

Choosing a Structure: The Complete Picture

Programming and Data Structures — Week 8, Lecture 4

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to state every major data structure covered in this course so far, and its associated operation costs, from a single consolidated table; choose the correct structure for a concretely described real workload from among all options covered, and justify that choice using this course's own proven results; explain the concrete distinction between "sorting algorithm," "search structure," and "priority structure" as three genuinely distinct problem categories, rather than interchangeable synonyms; and critique a real system design, identifying precisely which category-level mistake it makes, if any.

Outline

Today covers the complete toolbox this course has built so far, laid out in one place; a master comparison table consolidating every structure and every cost proven across eight weeks; the distinction between three genuinely different categories of problem — sorting, searching, and priority processing; five worked scenarios spanning every structure covered; a design review exercise identifying a category-level mistake in a realistic system; the MTech-track material on combining two structures for a single workload; an in-class quiz; and a summary.

The complete toolbox, laid out

Eight weeks have built a genuine toolbox, not a sequence of disconnected topics: the array from Week 2 provides direct index access; stacks and queues from Week 3 are disciplined interfaces layered over that array; linked lists from Week 4 trade contiguous memory for cheap endpoint operations; sorting algorithms from Weeks 5 and 6 impose a one-time or periodic full ordering on data; the balanced BST from Week 7 maintains that ordering continuously, adding range-query capability; and this week's hash table and heap each specialize further — the hash table for $O(1)$ average key lookup at the cost of all ordering, the heap for $O(1)$ minimum access at the cost of general search.

The master comparison table

This table consolidates every structure covered across the entire course against four operations. No row wins on every column — an unsorted array's $O(1)$ insertion comes at the cost of $O(n)$ search; a sorted array's $O(\log n)$ search comes at the cost of $O(n)$ insertion; a balanced BST wins broadly but with a larger constant factor than simpler structures; a hash table wins decisively on average-case search and insert but abandons ordering entirely; a heap wins decisively on minimum access specifically, at the cost of general search. Choosing correctly means matching this table's columns against a workload's actual, dominant access pattern — exactly the discipline built since Week 2 Lecture 4's array-versus-deque comparison.

Three categories of problem, not one

It is worth stating explicitly, as this course's closing structural lesson, that “sorting,” “searching,” and “priority processing” are three genuinely distinct categories of problem, not interchangeable synonyms for “organize my data.” Sorting means putting data into a full order, once or periodically — Weeks 5 and 6's algorithms. Searching means maintaining data so that any specific key can be found quickly and continuously as the data changes — Week 7's balanced BST or Week 8's hash table, depending on whether ordering is also required. Priority processing means repeatedly retrieving the single most urgent, smallest, or largest item as new data continuously arrives — Week 8's heap. Conflating these — using a full sort where a heap would suffice, or a hash table where range queries are actually needed — is exactly the category-level mistake Week 6 Lecture 4's leaderboard case study first identified, and this lecture's closing design review revisits directly.

Scenario 1: a spell-checker's dictionary of 500,000 words

A spell-checker's dictionary of 500,000 words needs to answer “is this word present?” extremely frequently, with no requirement whatsoever for ordering, range queries, or priority. A hash table is the clear choice: $O(1)$ average lookup directly serves the single dominant requirement, and none of the ordering capability a BST would add is ever used, making its extra overhead pure waste for this specific workload.

Scenario 2: an autocomplete feature showing suggestions in alphabetical order

An autocomplete feature that must show suggestions in alphabetical order for a given prefix requires answering “give me every word starting with this prefix, in sorted order” — genuinely a range query, not a plain membership lookup. A balanced BST answers this directly using Week 7 Lecture 4's range-query technique; a hash table, despite its excellent plain-lookup performance, cannot answer this efficiently at all, since it maintains no ordering among keys whatsoever. A specialized structure called a trie is

often used in real autocomplete systems for even better prefix-specific performance, but that structure is beyond this course's scope — the BST is the correct choice from what has been covered here.

Scenario 3: a job queue processing tasks by deadline

A job queue processing tasks by deadline needs to repeatedly retrieve and remove whichever task has the nearest deadline, while new tasks continuously arrive with arbitrary deadlines. This is exactly Lecture 3's priority queue pattern: a heap provides $O(\log n)$ insertion and $O(\log n)$ extraction of the minimum (nearest deadline), without paying for the full ordering capability a BST would provide but that this workload never actually needs.

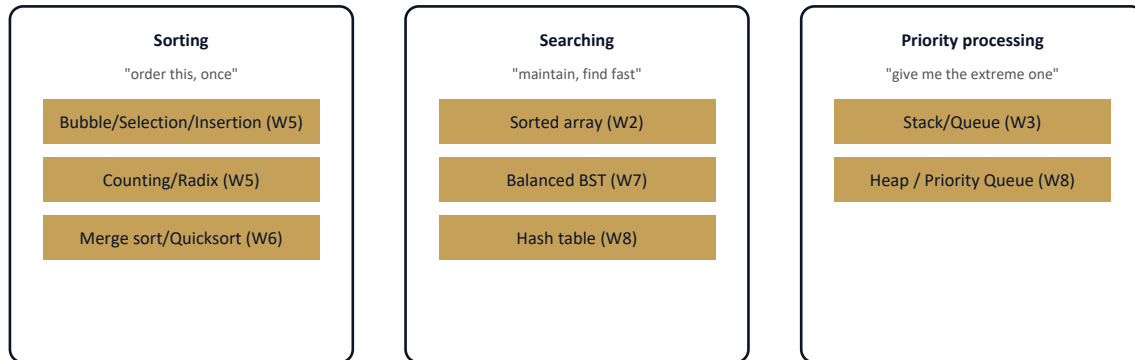
Scenario 4: a static list of 50 country codes, read constantly, never modified

A static list of 50 country codes, read constantly but never modified after being loaded once, does not need any of this course's more elaborate structures at all. A plain sorted array with Week 1 Lecture 4's binary search is the simplest and fastest choice: insertion cost is irrelevant, since insertion never happens after initial load, and a sorted array's $O(\log n)$ search, combined with minimal memory overhead and excellent cache locality, beats a hash table's or BST's added complexity for this specific, static workload.

Scenario 5: an undo history in a text editor

An undo history in a text editor needs only to add the most recent action and to remove and reapply the most recent action when undo is triggered — no other access pattern is ever needed. This is exactly Week 3's stack, the simplest structure in this entire course, and using anything more elaborate here would be solving a problem this workload does not have. If a "redo" capability is also required, needing bidirectional navigation between undone and redone actions, Week 4's doubly linked list (as in the browser-history application from that lecture) becomes the better choice instead.

The picture: the whole course's structures, mapped to categories



Balanced BST spans both searching and (via range queries) partial ordering — the only structure serving two categories at once.

This single diagram is worth retaining as the course's structural summary so far: every structure covered maps onto exactly one (or, for the balanced BST, two) of these three categories, and the correct starting question for any new workload is simply "which of these three things am I actually being asked to do?"

A design review: spotting the category mistake

This function fully sorts the entire ticket list on every single call, merely to read off the first 3 results — precisely the naive-design pattern Week 6 Lecture 4 first identified, now recognizable as a specific category mistake: using a sorting solution for what is genuinely a priority-processing problem. The correct fix is a small, fixed-size heap — holding only the current top 3 urgent tickets, maintained incrementally as tickets arrive or change — never re-sorting the entire dataset just to answer a query about a handful of extreme values. This is the same lesson as Week 6 Lecture 4's leaderboard case study, now made precise using this week's vocabulary: sorting and priority processing are different categories, and applying the wrong one wastes enormous, avoidable work.

MTech addition — when to combine two structures

Real systems very often need more than one access pattern simultaneously, and no single structure in this course's master table satisfies every column at once — this is not a gap in the table, but a genuine limitation of any single structure. The standard technique is combining two structures over the same logical data: Week 4 Lecture 4's LRU cache did exactly this, pairing a hash table (mapping keys directly to nodes, for $O(1)$ lookup) with a doubly linked list (threading those same nodes in recency order, for $O(1)$ reordering and eviction). Recognizing when a workload genuinely needs two different access patterns simultaneously, and combining two already-understood structures rather than searching for one perfect structure that does not exist, is itself an advanced and valuable design skill.

Quantifying the design review's fix

Quantifying the design review's fix with real numbers: the naive approach, sorting 10,000 tickets on every dashboard refresh, costs roughly $10,000 \times \log_2(10,000) \approx 130,000$ operations per call, using the same estimation technique from Week 6 Lecture 1. A size-3 max-heap maintained incrementally costs roughly $\log_2 3 \approx 2$ operations per ticket arrival or update, with reading the top 3 costing $O(1)$. The naive approach performs roughly 65,000 times more work per read than the correctly categorized fix — a difference of this magnitude is exactly why correctly identifying the problem's category, before choosing an algorithm within it, is the more consequential decision.

Exercise

As an exercise, devise three additional realistic scenarios of your own, apply this lecture's three-category framework to classify the actual problem each one poses, and choose an appropriate structure for each, justifying your choice directly against the master comparison table. Separately, sketch in a few sentences how you would combine a hash table and a heap to build a priority queue that additionally supports an $O(1)$ "does this item already exist in the queue?" check — a genuinely useful combination in scheduling systems that need to avoid duplicate work.

The course so far, as one continuous argument

It is worth seeing the whole course so far as one continuous argument, not eight separate topics. Week 1 established how to measure cost at all, through Big-O, and the class-based foundation everything else is built from. Weeks 2 through 4 covered storage structures, each trading a different access pattern for a different cost. Weeks 5 and 6 covered one-time ordering, arriving at increasingly better guaranteed costs, down to the proven $\Omega(n \log n)$ floor. Week 7 covered continuous ordering, actively maintained against the degeneration risk a naive approach would suffer. This week covered narrower but faster guarantees, earned specifically by deliberately abandoning ordering where it was not actually required. Each week's material is a direct, motivated response to a limitation exposed by the week before it.

Quiz — categorize and choose

Given a system that processes customer support tickets, always handling the single oldest unresolved ticket next, with new tickets arriving constantly, determine which of this lecture's three categories the problem belongs to and choose the appropriate structure, before checking the answer.

Quiz — answer

This is a priority-processing problem in the broad sense — repeatedly retrieving a single “most relevant” item as new items continuously arrive, not a full sort and not a general key-based search. But look more closely: “always the oldest unresolved ticket” is exactly Week 3’s FIFO discipline, needing no comparison beyond arrival order at all. A heap would technically work correctly here, but it is unnecessary machinery — Week 3’s plain queue already provides exactly the needed guarantee, at lower overhead. This is worth internalizing as a final, general caution: correctly identifying the category is necessary, but the *simplest* structure satisfying that category’s actual requirement is still the right engineering choice, not automatically the most sophisticated one covered in this course.

A final worked check: applying the framework cold

As a final check of the framework applied cold: a chat application needs to mark messages as read, show an unread count, and display messages in the order they were sent. Checking against this lecture’s three categories: “in the order sent” is naturally satisfied by storage order itself, requiring no separate sorting or search structure at all, since messages already arrive in order; “unread count” can often be maintained as a simple running counter, not requiring any structure whatsoever; and “mark as read” needs only $O(1)$ access to a specific message, which a plain array indexed by message ID (or a hash table if IDs are not small sequential integers) already provides. This scenario is a useful reminder that the correct answer is sometimes “no elaborate structure from this course’s toolbox is actually needed at all” — recognizing when a simpler structure fully suffices is as important as knowing the more advanced options exist.

Summary

Eight weeks have built a genuine, complete toolbox: arrays, stacks and queues, linked lists, sorting algorithms, balanced binary search trees, hash tables, and heaps, each with proven costs and a clear domain of applicability. Three genuinely distinct categories of problem — sorting, searching, and priority processing — each call for a different best-fit structure from that toolbox. The single most recurring mistake across every case study this course has used, from Week 6’s leaderboard to this lecture’s ticket dashboard, is applying the wrong *category* of solution entirely, not merely choosing a suboptimal algorithm within the correct category. Real systems frequently combine two structures over the same underlying data, each serving a different simultaneously-needed access pattern, as Week 4’s LRU cache demonstrated concretely. This closes the course’s core data structures unit. The remaining weeks turn to algorithmic techniques — dynamic programming, greedy algorithms, and graph algorithms — built on top of exactly these structures.